HONEYPOT CON COWRIE

Instalación, configuración y análisis de logs en Raspberry Pi

Tutorial completo · Seguridad defensiva · Lab académico

Este documento explica paso a paso cómo instalar Cowrie en una Raspberry Pi, qué hace cada comando, cómo interpretar los logs generados y qué conceptos de seguridad se aprenden en el proceso.

Cowrie 2.9.17 · Python 3.13.5 · Twisted 25.5.0 · Raspberry Pi OS (Debian 13)

1. ¿Qué es un Honeypot y para qué sirve?

Un **honeypot** (literalmente 'tarro de miel') es un sistema de seguridad diseñado para actuar como señuelo. Simula ser un servidor real y vulnerable con el objetivo de atraer a atacantes, registrar todo lo que hacen y aprender de sus técnicas sin poner en riesgo sistemas reales.

1.1 ¿Qué es Cowrie?

Cowrie es un honeypot de **SSH y Telnet** de código abierto. Cuando un atacante intenta conectarse, Cowrie simula ser un servidor Linux completamente funcional: acepta conexiones, responde a comandos y hace creer al atacante que está dentro de un sistema real. Mientras tanto, registra absolutamente todo en sus logs.

Lo que Cowrie captura de cada atacante:

- **Dirección IP** del atacante
- **Usuarios y contraseñas** que intenta
- **Comandos** que ejecuta una vez dentro
- **Archivos** que intenta descargar o subir
- **Herramientas** que intenta instalar (malware, scripts)
- **Huella digital SSH** del cliente (hassh fingerprint)
- **Duración** de cada sesión

1.2 ¿Por qué en Raspberry Pi?

La Raspberry Pi es ideal para un honeypot de aprendizaje: consume muy poca energía (puede estar encendida 24/7), tiene conectividad de red completa, corre Linux y su bajo costo hace que sea perfecta para un lab académico o doméstico.

Tipo de honeypot	Descripción	Ejemplo
SSH/Telnet honeypot	Simula servicios de acceso remoto	Cowrie
Web honeypot	Simula aplicaciones web vulnerables	Glastopf, DVWA
Base de datos honeypot	Simula bases de datos expuestas	HoneyMySQL
Red completa honeypot	Simula una red entera con múltiples servicios	OpenCanary, T-Pot

✔ Cowrie es uno de los honeypots más usados en investigación de seguridad y es mantenido activamente. Su repositorio en GitHub tiene miles de estrellas y la comunidad sigue activa.

2. Preparación del Sistema

Antes de instalar Cowrie se necesita preparar el sistema operativo y las dependencias. Esta sección explica cada paso y por qué es necesario.

2.1 Paso 1 — Actualizar el sistema

```
sudo apt update && sudo apt upgrade -y
```

¿Qué hace? apt update descarga la lista de paquetes disponibles desde los repositorios. apt upgrade instala las versiones más recientes de todo lo instalado. El flag -y confirma automáticamente.

¿Por qué hacerlo primero? Garantiza que no haya conflictos de versiones con las dependencias que instalaremos después, y que el sistema tenga todos los parches de seguridad.

2.2 Paso 2 — Instalar dependencias

```
sudo apt install git python3-virtualenv libssl-dev libffi-dev \
  build-essential libpython3-dev python3-minimal authbind virtualenv -y
```

Paquete	Para qué sirve
git	Descargar el código fuente de Cowrie desde GitHub
python3-virtualenv / virtualenv	Crear entornos virtuales aislados de Python
libssl-dev	Librería de desarrollo de OpenSSL — necesaria para compilar cryptography
libffi-dev	Foreign Function Interface — permite que Python llame código C nativo
build-essential	Compilador GCC y herramientas de construcción (make, etc.)
libpython3-dev	Headers de Python 3 para compilar extensiones nativas
authbind	Permite que usuarios no-root escuchen en puertos menores a 1024 (como el 22)

2.3 Paso 3 — Crear usuario dedicado para Cowrie

```
sudo adduser --disabled-password cowrie
```

¿Por qué es crítico? Este es uno de los principios más importantes de seguridad: **principio de mínimo privilegio**. Cowrie nunca debe correr como root. Si un atacante logra escapar del honeypot (aunque Cowrie está diseñado para evitarlo), el daño queda limitado a lo que puede hacer el usuario cowrie, que no tiene privilegios.

■ NUNCA corras Cowrie como root. Un honeypot comprometido con root daría al atacante control total del sistema real.

2.4 Paso 4 — Cambiar al usuario cowrie y clonar el repositorio

```
sudo su - cowrie
git clone http://github.com/cowrie/cowrie
cd cowrie
```

sudo su - cowrie: cambia al usuario cowrie con su entorno completo (variables de entorno, directorio home, etc.). El guión - es importante — carga el entorno completo.

git clone: descarga el repositorio completo de Cowrie desde GitHub incluyendo todo el historial de cambios. Crea el directorio /home/cowrie/cowrie/.

3. Instalación con Entorno Virtual Python

3.1 ¿Qué es un entorno virtual de Python?

Un entorno virtual es una instalación aislada de Python con sus propias librerías, completamente separada del Python del sistema operativo. Esto evita conflictos entre versiones: si el sistema usa cryptography 40.x y Cowrie necesita cryptography 46.x, coexisten sin problemas.

3.2 Paso 5 — Crear e instalar el entorno virtual

```
virtualenv --python=python3 cowrie-env
source cowrie-env/bin/activate
```

virtualenv --python=python3 cowrie-env: crea el entorno virtual en el directorio cowrie-env/. La opción --python=python3 fuerza el uso de Python 3 (no Python 2).

source cowrie-env/bin/activate: activa el entorno. Después de esto, el prompt muestra (cowrie-env) y todos los comandos python y pip usan el entorno aislado.

> ✔ Una vez activado el entorno, verás (cowrie-env) al inicio del prompt. Esto confirma que estás dentro del entorno virtual.

```
pip install --upgrade pip
pip install -r requirements.txt
```

pip install --upgrade pip: actualiza el gestor de paquetes dentro del entorno virtual.

pip install -r requirements.txt: lee el archivo requirements.txt que lista todas las librerías necesarias con sus versiones exactas, y las instala.

3.3 Librerías principales que se instalan

Librería	Versión	Función en Cowrie
twisted	25.5.0	Framework de red asíncrono — el motor central de Cowrie para manejar conexiones SS
cryptography	46.0.6	Implementación de algoritmos criptográficos — cifrado SSH, curvas elípticas, etc.
bcrypt	5.0.0	Hashing de contraseñas para la base de datos de usuarios del honeypot
pyasn1	0.6.3	Manejo de estructuras ASN.1 usadas en certificados SSH y X.509
service_identity	24.2.0	Verificación de identidad en conexiones TLS/SSL
treq	25.5.0	Cliente HTTP asíncrono para que Cowrie pueda hacer peticiones salientes
requests	2.32.5	Librería HTTP para integraciones y output plugins
tftpy	0.8.7	Implementación del protocolo TFTP para simular transferencias de archivos

3.4 Paso 6 — Instalar Cowrie como comando ejecutable

```
pip install -e .
```

Este comando instala Cowrie en modo 'editable' (**-e**) usando el archivo pyproject.toml. Esto hace dos cosas importantes:

- Registra el comando cowrie en el PATH del entorno virtual, permitiendo usar cowrie start, cowrie stop, etc.
- En modo editable, cualquier cambio al código fuente en src/ se refleja sin reinstalar

■ La guía original indicaba usar bin/cowrie start, pero en versiones modernas de Cowrie (2.x con pyproject.toml) el binario se instala via pip. El comando correcto es cowrie start después de pip install -e .

4. Configuración de Cowrie

4.1 Estructura del archivo de configuración

```
cp etc/cowrie.cfg.dist etc/cowrie.cfg
nano etc/cowrie.cfg
```

Cowrie incluye un archivo de ejemplo cowrie.cfg.dist con todas las opciones comentadas. Lo copiamos a cowrie.cfg para editarlo sin modificar el original.

4.2 Parámetros clave en [honeypot]

```
[honeypot]
hostname = webserver01
listen_port = 2222
```

Parámetro	Valor usado	Por qué importa
hostname	webserver01	Nombre que verán los atacantes al entrar. Debe ser creíble: 'server01', 'ubuntu-prod', '
listen_port	2222	Puerto donde escucha Cowrie. No usar el 22 directamente (requeriría ser root). Se pue

■ Error encontrado durante la instalación: al editar cowrie.cfg se duplicó la opción hostname en la sección [honeypot]. Python configparser no permite opciones duplicadas y lanza DuplicateOptionError. Solución: comentar con # la línea duplicada.

4.3 Error que ocurrió y su solución

```
configparser.DuplicateOptionError: While reading from
'/home/cowrie/cowrie/etc/cowrie.cfg' [line 30]:
option 'hostname' in section 'honeypot' already exists
```

Causa: al editar el archivo de configuración se añadió una línea hostname = webserver01 sin eliminar la que ya existía.

Solución: abrir el archivo con nano, buscar la sección [honeypot] y comentar una de las dos líneas con #:

```
[honeypot]
# hostname = svr04      <-- comentar la original
hostname = webserver01  <-- dejar solo esta
```

5. Inicio y Verificación del Honeypot

5.1 Iniciar Cowrie

```
cowrie start
```

Este comando lanza Cowrie en segundo plano usando **twistd** (el daemon de Twisted). La salida mostrará advertencias sobre TripleDES — son completamente normales:

```
Starting cowrie: [twistd --umask=0022 --pidfile .../cowrie.pid ...]
CryptographyDeprecationWarning: TripleDES has been moved to
cryptography.hazmat.decrepit.ciphers.algorithms.TripleDES...
```

¿Por qué el warning de TripleDES? Twisted sigue incluyendo soporte para el algoritmo 3DES-CBC en su implementación SSH por compatibilidad con clientes antiguos. La librería cryptography avisó que moverá ese algoritmo a un módulo 'decrepit' en versiones futuras. No afecta el funcionamiento actual.

5.2 Comandos de gestión

Comando	Función
cowrie start	Inicia Cowrie en segundo plano como daemon
cowrie stop	Detiene el proceso Cowrie
cowrie status	Muestra si está corriendo y el PID del proceso
cowrie restart	Detiene e inicia (útil después de cambiar configuración

5.3 Verificar que está escuchando en el puerto

```
cowrie status
# Resultado: cowrie is running (PID: 39186)

netstat -tlnp | grep 2222
# Resultado:
# tcp  0  0 0.0.0.0:2222  0.0.0.0:*  LISTEN  39186/python
```

Columna de netstat	Significado
tcp	Protocolo de transporte
0 0.0.0.0:2222	Escucha en todas las interfaces de red, puerto 2222
0.0.0.0:*	Acepta conexiones desde cualquier IP remota
LISTEN	Estado: esperando conexiones entrantes
39186/python	PID del proceso y nombre del ejecutable

El mensaje *'Not all processes could be identified, non-owned process info will not be shown, you would have to be root to see it all'* es normal cuando netstat se ejecuta sin root — solo puede mostrar los procesos del usuario actual.

5.4 Probar la conexión al honeypot

```
ssh -p 2222 root@localhost
```

Al conectarse con cualquier contraseña (en la configuración por defecto, contraseña vacía funciona para root), Cowrie acepta la conexión y muestra un shell falso:

```
root@webserver01:~#
```

El hostname webserver01 es exactamente el que configuramos — el atacante cree que está dentro de un servidor real llamado 'webserver01'. Todo lo que escriba en ese shell será registrado.

> ✔ El intento con 'testuser' falló porque Cowrie por defecto solo acepta root con contraseña vacía. Para personalizar qué usuarios y contraseñas acepta, se crea el archivo etc/userdb.txt con el formato: usuario:contraseña

6. Logs — Interpretación Completa

Los logs de Cowrie son la parte más valiosa del honeypot. Aquí se registra todo lo que hace cada atacante. Cowrie genera dos archivos de log en var/log/cowrie/:

- **cowrie.log**: formato de texto legible por humanos, con timestamps UTC
- **cowrie.json**: formato JSON estructurado, ideal para análisis automatizado con herramientas como Elasticsearch, Splunk o scripts Python

6.1 ¿Dónde estaban los logs?

```
# INCORRECTO (ruta de versiones antiguas):
var/log/cowrie.log

# CORRECTO (versiones modernas 2.x):
var/log/cowrie/cowrie.log
var/log/cowrie/cowrie.json
```

■ Los logs no se crean hasta que hay la primera conexión al honeypot. Si el archivo no existe aún, es normal — simplemente no ha habido actividad todavía.

6.2 Anatomía de una entrada de log

```
2026-04-16T00:53:32.808686Z [-] CowrieSSHFactory starting on 2222
2026-04-16T00:53:33.598626Z [-] Ready to accept SSH connections
2026-04-16T00:57:18.222523Z [CowrieSSHFactory] New connection: 127.0.0.1:35514 (127.0.0.1:2222) [session:
2026-04-16T00:57:18.228280Z [HoneyPotSSHTransport,0,127.0.0.1] Remote SSH version: SSH-2.0-OpenSSH_10.0p2
2026-04-16T00:57:18.235522Z [HoneyPotSSHTransport,0,127.0.0.1] SSH client hassh fingerprint: eeca2460550b
2026-04-16T00:57:37.310248Z [HoneyPotSSHTransport,0,127.0.0.1] login attempt [b'root'/b''] succeeded
2026-04-16T00:57:37.316745Z [HoneyPotSSHTransport,0,127.0.0.1] Initialized emulated server as architectur
```

Elemento del log	Descripción
2026-04-16T00:57:18.222523Z	Timestamp en formato ISO 8601, zona UTC (Z = UTC+0)
[CowrieSSHFactory]	Componente interno de Cowrie que generó el evento
New connection: 127.0.0.1:35514	IP y puerto del cliente que se conectó
(127.0.0.1:2222)	IP y puerto local de Cowrie que recibió la conexión
[session: faefea258fb9]	ID único de esta sesión — permite rastrear todos los eventos de una misma conexión
Remote SSH version: SSH-2.0-OpenSSH_10.0p2	Cliente SSH que usa el atacante — puede revelar su SO
hassh fingerprint: eeca246...	Hash del conjunto de algoritmos SSH del cliente. Permite identificar el mismo tool en mú
login attempt [b'root'/b'']	Usuario y contraseña del intento. b'' = cadena bytes vacía = contraseña vacía

succeeded / failed	Si el intento fue aceptado por la base de datos de usuarios de Cowrie
Initialized emulated server as architecture: linux-x64-lsb	Cowrie preparó el shell simulado para esa arquitectura

6.3 Interpretación de tu log completo

Analizando los logs generados durante las pruebas, esto es lo que ocurrió:

Hora (UTC)	Evento	Interpretación
00:53:31	Cowrie inicia y lee configuración	Arranque correcto. Lee cowrie.cfg.dist + cowrie.cfg
00:53:32	Python 3.13.5, Twisted 25.5.0, Cowrie 2.9.17	Versiones del entorno confirmadas en log
00:53:32	CowrieSSHFactory starting on 2222	Servidor SSH falso activo en puerto 2222
00:57:18	Nueva conexión sesión faefea258fb9 desde 127.0.0.1	Tu prueba manual con ssh root@localhost
00:57:18	hassh: eeca2460...	Huella del cliente SSH (OpenSSH 10.0p2 de Debian)
00:57:37	login attempt [root/''] succeeded	Root con contraseña vacía — aceptado por la DB por defecto
00:57:37	Initialized emulated server linux-x64-lsb	Cowrie preparó shell Linux 64-bit simulado
00:58:16	Nueva conexión sesión d4d6b210e095	Segunda prueba con testuser
00:58:23	login attempt [testuser/''] failed	testuser no está en la DB por defecto — rechazado
01:00:37	TTY Log cerrado, avatar root logging out	Sesión root cerrada después de 199 segundos
01:07:58	Timeout reached in HoneyPotSSHTransport	Sesión sin actividad cerrada automáticamente por timeout

6.4 El TTY Log — grabación de sesiones

```
Closing TTY Log: var/lib/cowrie/tty/e3b0c44298fc1c149afbf4c8996fb924... after 180.0 seconds
```

Cowrie graba la sesión completa en formato TTY en var/lib/cowrie/tty/. Cada archivo es una grabación de todo lo que ocurrió en el terminal: comandos escritos, output recibido, caracteres de control. Se puede reproducir con:

```
# Reproducir una sesión grabada:
cowrie playlog var/lib/cowrie/tty/e3b0c44298fc...
```

✔ El nombre del archivo TTY es el hash SHA-256 del contenido. Si el hash es e3b0c44298fc1c149afbf4c8996fb924... (el hash SHA-256 de un string vacío), significa que la sesión estuvo conectada pero no se ejecutó ningún comando.

7. Estructura Completa de Archivos

```
/home/cowrie/cowrie/
■■■ cowrie-env/              # Entorno virtual Python (NO tocar manualmente)
■   ■■■ bin/                 # Ejecutables: python, pip, cowrie, twistd
■   ■■■ lib/python3.13/      # Librerías instaladas
■■■ etc/
■   ■■■ cowrie.cfg.dist      # Configuración de ejemplo (NO editar)
■   ■■■ cowrie.cfg           # Configuración activa (ESTE se edita)
■■■ var/
■   ■■■ log/cowrie/
■   ■   ■■■ cowrie.log       # Logs legibles por humanos
■   ■   ■■■ cowrie.json      # Logs estructurados JSON
■   ■■■ lib/cowrie/tty/      # Grabaciones de sesiones TTY
■   ■■■ run/cowrie.pid       # PID del proceso en ejecución
■■■ src/cowrie/              # Código fuente Python de Cowrie
■   ■■■ core/                # Núcleo: autenticación, configuración
■   ■■■ ssh/                 # Implementación del protocolo SSH
■   ■■■ output/              # Plugins de salida (JSON, MySQL, etc.)
■■■ honeyfs/                 # Sistema de archivos falso que ve el atacante
■   ■■■ etc/passwd           # /etc/passwd falso con usuarios simulados
■■■ bin/                     # Scripts auxiliares (NO el binario principal)
■■■ requirements.txt         # Lista de dependencias Python
■■■ pyproject.toml           # Configuración del paquete Python moderno
```

Directorio/Archivo	Para qué sirve	¿Se modifica?
etc/cowrie.cfg	Configuración principal del honeypot	Sí — para personalizar
var/log/cowrie/	Logs de actividad	No — los genera Cowrie automáticamente
var/lib/cowrie/tty/	Grabaciones TTY de sesiones	No — los genera Cowrie automáticamente
honeyfs/	Sistema de archivos simulado para los atacantes	Sí — para hacerlo más realista
src/cowrie/	Código fuente	Solo si se modifica el código de Cowrie
cowrie-env/	Entorno virtual Python	Nunca directamente

8. Configuración Avanzada y Próximos Pasos

8.1 Redirigir el puerto 22 al honeypot

Para capturar atacantes que escaneen el puerto SSH estándar (22), se redirige el tráfico con iptables:

```
# Redirigir puerto 22 -> 2222 (donde escucha Cowrie)
sudo iptables -t nat -A PREROUTING -p tcp --dport 22 -j REDIRECT --to-port 2222

# Hacer la regla permanente:
sudo apt install iptables-persistent
sudo netfilter-persistent save
```

¿Cómo funciona? iptables opera en el kernel de Linux interceptando paquetes de red. La tabla nat con la cadena PREROUTING intercepta paquetes antes de que sean procesados. El destino REDIRECT cambia el puerto destino al 2222.

■ Si tu SSH real también está en el puerto 22, esta regla te desconectará. Primero cambia el SSH real a otro puerto (ej. 22222) en /etc/ssh/sshd_config, luego aplica la redirección.

8.2 Personalizar usuarios y contraseñas aceptadas

```
# Crear base de datos de usuarios personalizada:
nano etc/userdb.txt

# Formato: usuario:contraseña (una por línea)
# Usar * para aceptar cualquier contraseña
root:*
admin:admin
ubuntu:ubuntu
pi:raspberry
```

Con este archivo, Cowrie aceptará conexiones con esas credenciales exactas. Las credenciales comunes (admin/admin, root/root, pi/raspberry) son las que los atacantes intentan primero en ataques de diccionario.

8.3 Hacer que Cowrie inicie automáticamente al arrancar

```
# Opción 1: crontab
sudo crontab -e
# Añadir:
@reboot sudo su - cowrie -c 'cd /home/cowrie/cowrie && source cowrie-env/bin/activate && cowrie start'

# Opción 2: systemd service (más robusto)
sudo nano /etc/systemd/system/cowrie.service
```

```
[Unit]
Description=Cowrie SSH Honeypot
After=network.target
```

```
[Service]
Type=forking
User=cowrie
WorkingDirectory=/home/cowrie/cowrie
ExecStart=/home/cowrie/cowrie/cowrie-env/bin/cowrie start
ExecStop=/home/cowrie/cowrie/cowrie-env/bin/cowrie stop

[Install]
WantedBy=multi-user.target
```

```
sudo systemctl daemon-reload
sudo systemctl enable cowrie
sudo systemctl start cowrie
```

8.4 Ver logs en tiempo real

```
tail -f var/log/cowrie/cowrie.log

# Solo los intentos de login:
grep 'login attempt' var/log/cowrie/cowrie.log

# Solo los logins exitosos:
grep 'succeeded' var/log/cowrie/cowrie.log

# IPs únicas que se conectaron:
grep 'New connection' var/log/cowrie/cowrie.log | awk '{print $8}' | sort -u
```

8.5 Análisis del archivo JSON

```
# Ver el JSON formateado:
cat var/log/cowrie/cowrie.json | python3 -m json.tool | head -50

# Extraer todas las IPs y contraseñas probadas:
cat var/log/cowrie/cowrie.json | python3 -c "
import sys,json
for line in sys.stdin:
    try:
        d=json.loads(line)
        if d.get('eventid')=='cowrie.login.failed':
            print(f\"{d.get('src_ip')} tried {d.get('username')}:{d.get('password')}\")
    except: pass
"
```

9. Conceptos de Seguridad que se Aprenden

Instalar y operar un honeypot no es solo un ejercicio técnico — enseña conceptos fundamentales de seguridad defensiva que aplican en cualquier entorno profesional.

9.1 Principio de mínimo privilegio

Cowrie corre con su propio usuario sin privilegios. Este principio establece que cualquier proceso, usuario o sistema debe tener exactamente los permisos que necesita para funcionar, y nada más. Limita el daño en caso de compromiso.

9.2 Cómo funcionan los ataques de fuerza bruta SSH

Los logs de Cowrie muestran exactamente cómo opera un ataque de diccionario SSH:

- El atacante escanea Internet buscando puertos 22 abiertos (con masscan o zmap)
- Para cada IP encontrada, prueba automáticamente combinaciones usuario/contraseña comunes
- Los más probados: root/root, root/123456, admin/admin, pi/raspberry, ubuntu/ubuntu
- Todo ocurre en segundos — scripts automatizados prueban miles de combinaciones por hora

9.3 Protocolo SSH — lo que revelan los logs

Elemento SSH	Lo que revela en los logs de Cowrie
SSH version string	Sistema operativo y versión del cliente del atacante
KEX (Key Exchange) algorithm	Curva25519-sha256 indica cliente moderno; kex antiguo puede indicar herramienta específica
HASSH fingerprint	Hash reproducible del cliente SSH — identifica la misma herramienta en diferentes IPs
auth b'none' primero	El cliente SSH siempre intenta autenticación 'none' primero para obtener la lista de métodos p
PTY request (xterm)	El atacante pidió un terminal interactivo — quiere una shell, no solo ejecutar un comando
no-more-sessions@openssh.com	El cliente OpenSSH moderno indica que no abrirá más sesiones en esta conexión

9.4 Threat Intelligence con honeypots

Los datos recolectados por Cowrie tienen valor real de inteligencia de amenazas:

- **Mapeo de TTPs** (Tactics, Techniques & Procedures): qué comandos ejecutan los atacantes una vez dentro — revela sus objetivos (instalar miners, backdoors, bots)
- **Threat feeds**: las IPs capturadas por Cowrie pueden alimentar listas de bloqueo
- **Malware analysis**: si un atacante sube un archivo, Cowrie lo guarda para análisis

• **Credential databases**: las contraseñas más probadas forman diccionarios útiles para auditar sistemas propios

✔ El proyecto Cowrie comparte datos de forma anonimizada con la comunidad de seguridad. Puedes contribuir tus datos en cowrie.org si tu honeypot está expuesto a Internet real.

10. Referencia Rápida — Comandos del Día a Día

10.1 Secuencia completa de inicio desde cero

```
# 1. Cambiar al usuario cowrie
sudo su - cowrie

# 2. Ir al directorio
cd cowrie

# 3. Activar entorno virtual
source cowrie-env/bin/activate

# 4. Iniciar honeypot
cowrie start

# 5. Verificar que corre
cowrie status

# 6. Verificar puerto
netstat -tlnp | grep 2222
```

10.2 Gestión diaria

Tarea	Comando
Iniciar el honeypot	cowrie start
Detener el honeypot	cowrie stop
Ver estado	cowrie status
Reiniciar tras cambios de config	cowrie restart
Ver logs en tiempo real	tail -f var/log/cowrie/cowrie.log
Ver solo intentos de login	grep 'login attempt' var/log/cowrie/cowrie.log
Ver logins exitosos	grep 'succeeded' var/log/cowrie/cowrie.log
Ver IPs conectadas hoy	grep 'New connection' var/log/cowrie/cowrie.log | grep $(date +%Y-%m-%d)
Apagar la Raspberry Pi	sudo shutdown -h now
Ver logs JSON formateados	cat var/log/cowrie/cowrie.json | python3 -m json.tool

10.3 Verificación de que todo funciona — checklist

Verificación	Comando	Resultado esperado
Proceso activo	cowrie status	cowrie is running (PID: XXXXX)
Puerto escuchando	netstat -tlnp | grep 2222	tcp 0.0.0.0:2222 LISTEN
SSH conecta	ssh -p 2222 root@localhost	root@webserver01:~#
Logs se generan	ls var/log/cowrie/	cowrie.log y cowrie.json presentes
Login registrado	grep 'login attempt' var/log/cowrie/cowrie.log	Aparece el intento

---

Documento generado a partir de la instalación real de Cowrie 2.9.17 en Raspberry Pi. Python 3.13.5 · Twisted 25.5.0 · Raspberry Pi OS (Debian 13) · Uso académico en laboratorio.